

Reth ExEx using gRPC

A modular approach

Zidong Zhang

ExEx Recap

An Execution Extension is a task that listen to and runs on certain blockchain state change events (i.e. commit/reorg/revert)



WHAT

PoC to build Reth ExEx that can depend on:

- **ANY external systems**
- **writing in ANY language**

WHY

- **Code reusability (not reinvent the wheels)**
- **Modularity (loosely coupled systems; micro service)**
- **Flexibility (devs have diff language choice)**

HOW

- **Run subprocess in a shell**
- **RPC**
- **gRPC**
- **...**

Why gRPC:

- 1. type safety**
- 2. strong multi-language support**
- 3. simple**

PoC sys overview

- **protobuf: 3 handlers**
- **grpc:**
 - **gRPC server implemented using Go**
 - **prints the requests data**
- **exex:**
 - **reth with ExEx installed (docker)**
 - **auto-generate gRPC client with github.com/hyperium/tonic**
 - **gRPC call inside the ExEx**
- **demo: private testnet using previously built reth image**



github.com/bap2pecs/reth-exex-grpc-example

Summary

- The gRPC handlers definition are flexible
- “ExExes SHOULD emit an ExExEvent::FinishedHeight event to signify what blocks have been processed. This event is used by Reth to determine what state can be pruned.”
- built.rs + /proto + Cargo.toml (tonic)
 - Rust gRPC client will be built inside /target
 - Use the client in your ExEx
- gRPC server (language agnostic)

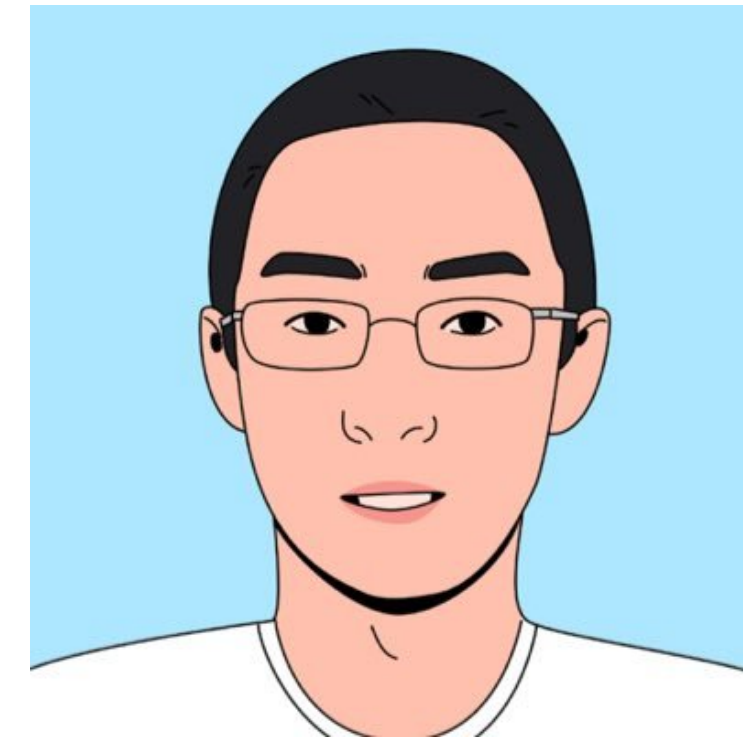
References

- <https://www.paradigm.xyz/2024/05/reth-exex>
- <https://reth.rs/run/private-testnet.html>
- <https://reth.rs/installation/docker.html#building-the-docker-image>
- <https://reth.rs/developers/exex/exex.html>
- <https://github.com/paradigmxyz/reth-exex-examples/tree/main/minimal>
- <https://github.com/paradigmxyz/reth/blob/main/Dockerfile>
- <https://github.com/hyperium/tonic>
- <https://docs.kurtosis.com/quickstart>

Q&A

TG: @zidongz

X: @0xb2p



github.com/bap2pecs/reth-exex-grpc-example